

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконала: студентка групи AI-243
Кравченко В. О.

Перевірив: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпоінт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	654	10
Час, с	101,71	1,61
Токенів/с	7,62	20,17

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1145,6	177,2
Час, с	137,32	13,99

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: У qwen3:4b – у поле `thinking`. Response короткий (5 грн), а `thinking` – величезний текст на 953–1395 токенів, де модель розписує рівняння, перевіряє себе, сумнівається. У llama3.2:3b поля `thinking` взагалі немає – усі токени пішли одразу у `response`.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі значно перевершують звичайні на складних задачах, які вимагають багатокрокової логіки: математика, програмування,

планування та аналіз . Результат qwen3:4b дала 5/5 правильних відповідей на логічну задачу, тоді як звичайна llama3.2:3b – лише 2/5

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі навчають на прикладах «питання-відповідь» та винагороджують за правильну кінцеву відповідь . Міркувальні моделі додатково проходять етап навчання з підкріпленням, де винагороджується не лише правильна відповідь, а й якість самого процесу міркування (перевірка, виправлення помилок)

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, не означає. Це означає, що модель витрачає більше обчислювальних ресурсів на «роздуми» для підвищення точності. Можна навчити модель генерувати багато токенів, але якщо вони не ведуть до правильної відповіді, це марна трата. Наприклад, llama3.2:3b витратила менше токенів, але часто помилялася, а qwen3:4b витратила багато, але дала правильну відповідь.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різниця пояснюється двома факторами: режимом роботи та архітектурою. По-перше, qwen3:4b перебувала в режимі міркування, тому генерувала значно більше токенів, що займає більше часу. По-друге, навіть швидкість генерації токенів у неї була нижчою. Це може бути пов'язано з розміром моделі та ефективністю інференсу.

Міркувальна модель доцільна там, де ціна помилки вища за ціну часу – багатокрокові розрахунки, планування, код, де тихе вгадування відповіді часто веде до неправильного результату, як показала llama3.2:3b (2/5). Інструктивна модель натомість виграє там, де відповідь очевидна й важлива швидкість – прості факти, короткі формати, інтерактивні чат-боти, де користувач не готовий чекати хвилини заради того самого результату. На практиці розумний вибір – не яка модель краща", а перемикання режиму під тип задачі: якщо у мене є класифікатор задач, я б рутинні й фактичні запити віддавала інструктивній моделі, а складні розрахунки чи агентські ланцюжки – міркувальній, приймаючи ціну в часі свідомо.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 63 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 1098,6 с проти 111,9 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Виконано по два прогони для кожної з двох моделей (llama3.2:3b та qwen3:4b) з різними system-повідомленнями. Запит користувача (user) однаковий: «Поясни різницю між компілятором та інтерпретатором». Перше system-повідомлення – нейтральне (Ти лаконічний технічний асистент), друге – жорстке (Відповідай лише українською, максимум двома реченнями).

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	2537	змішана (укр, рус, англ)	~28	ні
llama3.2:3b	українською, 2 речення	289	українська (з помилками)	2	частково
qwen3:4b	лаконічний асистент	235	англійська	2	частково
qwen3:4b	українською, 2 речення	179	українська	2	так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	55	477	532
llama3.2:3b	українською, 2 речення	59	74	133
qwen3:4b	лаконічний асистент	59	2870	2929
qwen3:4b	українською, 2 речення	68	930	998

Відповіді на питання (крок 6):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві моделі виконали обмеження, але qwen3:4b зробила це якісніше – 2 речення, чиста українська, зміст точний. Llama3.2:3b теж уклалася в 2 речення, але з грубими мовними помилками

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Прямої вказівки українською не було в прогонах 1 і 3, тому llama3.2:3b у прогоні 1 змішала мови (укр, рус, англ), а qwen3:4b у прогоні 3 відповіла повністю англійською. Це показує: без явної вказівки мови модель обирає англійську як мову за замовчуванням.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: prompt_tokens зросла, бо нове system-повідомлення довше. Кожен токен інструкції додається до контексту.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: Бо qwen3:4b – міркувальна модель: вона генерує внутрішню чернетку, яка рахується в completion_tokens, але не показується користувачу. Llama3.2:3b – інструктивна, вона одразу видає фінальну відповідь.

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п'ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Моделі навчені на різних даних і з різними стратегіями вирівнювання. llama3.2:3b має слабший instruction-following через малий розмір і обмежений україномовний корпус – вона зривається на домінантні мови. qwen3:4b краще тримає інструкції завдяки reasoning-тренуванню, але без вказівки мови обирає англійську як мову за замовчуванням.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі – чи справді воно має більший пріоритет?

Відповідь: Технічно – нічим, це просто текст у контексті. Але під час навчання модель бачила, що system-повідомлення зазвичай містить правила гри, тому надає йому вищий пріоритет. Це статистична схильність, а не жорстке правило. Якщо user-інструкція конкретніша й з'явилася пізніше – вона може переважити.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Бо reasoning-модель генерує проміжні кроки міркування (thinking) перед фінальною відповіддю. Ці кроки не видно в response, але вони рахуються в completion_tokens. Це плата за кращу якість на складних задачах – але на простих питаннях це чистий оверхед.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, - яку вона виконає?

Відповідь: Найімовірніше, user-інструкцію, якщо вона конкретніша й з'явилася пізніше в контексті. Але поведінка недетермінована: іноді модель застрягає й видає щось середнє, або ігнорує обидві.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Бо в навчальному корпусі малих моделей української мови мало, а російської та англійської – багато. Модель переключається на домінантну мову,

особливо під час генерації складних конструкцій. Великі моделі мають більше параметрів і краще узагальнюють мовні інструкції.

System-повідомлення – це найдешевший важіль керування моделлю: не потрібно ні донавчання, ні зміни коду. Але його надійність обмежена. Прогони показали: інструкція по довжині виконується стабільно, а по мові – ні. Llama3.2:3b з прямою вказівкою українською все одно видала суржик, а qwen3:4b без вказівки мови пішла в англійську.

Висновок за завданням 2

Експеримент показав, що system-повідомлення – це найдешевший важіль керування моделлю: не потрібно ні до навчання, ні зміни коду, достатньо одного рядка тексту. Але його надійність обмежена й залежить від моделі. Інструкція по довжині виконується стабільно обома моделями, а інструкція по мові – ні: Llama3.2:3b з прямою вказівкою українською все одно видала суржик із русизмами, а qwen3:4b без вказівки пішла в англійську. Нейтральні інструкції типу лаконічний асистент не працюють узагалі – обидві моделі проігнорували їх. Практичний висновок: system-повідомлення – необхідний, але не достатній інструмент. Якщо модель ігнорує інструкцію, треба продублювати її в user-повідомленні, використати програмні обмеження або взяти більшу модель. Для українськомовних задач qwen3:4b показала себе значно краще за Llama3.2:3b: вона єдина дала чисту українську без русизмів і кальок. Ціна цього – у 6–12 разів більше completion_tokens через режим міркування, що для коротких відповідей є суттєвою перевитратою.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Виконано запит до двох моделей через єдиний OpenAI-сумісний інтерфейс: локальної Llama3.2:3b і хмарної openai/gpt-oss-120b. Ключ доступу збережено у файлі .env, який не потрапляє в Git завдяки .gitignore. Запит користувача однаковий для обох: Поясни різницю між компілятором та інтерпретатором. Для кожного

прогону зафіксовано час відповіді, видимий текст, мову, наявність фактичних помилок і показники `response.usage`.

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	23,80	0,92
completion_tokens	153	238
Довжина відповіді, символів	518	581
Мова відповіді	суржик (укр+рус)	українська
Фактичних помилок у відповіді	4	0

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Фактично жодного рядка в логіці. Змінилися лише два значення у словнику PROVIDERS: `base_url` і `model`. Сам виклик ідентичний для обох.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна `openai/gpt-oss-120b` – значно точніша. Вона правильно описує: компілятор одноразово переводить код у машинний, створюючи окремий виконуваний файл; інтерпретатор читає код построчно під час виконання. Локальна `llama3.2:3b` вигадує неіснуючі терміни – «запам'ятовувальна система», і тд., які не мають сенсу.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, багато. По-перше, мова суржик із русизмами. По-друге, вигадані терміни.

4. Хмарна модель відповіла швидше за локальну – за рахунок чого, якщо запит летить через інтернет?

Відповідь: Groq використовує спеціалізоване обладнання LPU, оптимізоване саме під інференс мовних моделей. Швидкість генерації сягає ~1000 токенів/с, що на порядок перевищує CPU-інференс локальної моделі. Затримка – мізерна порівняно з вирашем у швидкості обчислень.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першими задали де-факто стандарт API для LLM, і на момент появи конкурентів їхній формат уже був добре відомий розробникам. Провайдерам було вигідніше підтримати сумісність, ніж змушувати клієнтів переписувати код. Практичний наслідок один і той самий код працює і з Ollama, і з Groq, і з Google, змінюється лише адреса й ключ.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір важливий, але не єдиний фактор. Ключову роль відіграють якість навчальних даних, стратегія вирівнювання, архітектура та тренування на конкретних мовах. Наш експеримент показав: 3B-модель провалює українську не стільки через розмір, скільки через брак україномовних даних у корпусі.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Groq використовує спеціалізовані LPU-чипи, оптимізовані під інференс LLM. Вони забезпечують генерацію ~1000 токенів/с, тоді як CPU-інференс дає ~5–10 токенів/с. Затримка мережі (50–200 мс) на цьому тлі незначна.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Усі дані, які ви надсилаєте, потрапляють на чужий сервер і можуть зберігатися для навчання чи аудиту. У комерційних проєктах використовують: корпоративні контракти з гарантією «no training on your data», локальні моделі для

чутливих даних, маскуванню/анонімізацію РІІ перед відправкою, self-hosted рішення для критичних систем.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit – це обмеження на кількість запитів або токенів за одиницю часу. Розробники обходять це через: черги й експоненційний backoff при помилці 429, ротацію між кількома провайдерами, кешування однотипних запитів, платні тарифи для продакшену

Локальна модель варта вибору там, де дані не можна передавати назовні, де потрібна робота без інтернету, або для навчання й прототипування, коли якість не критична. Вона безкоштовна, приватна й не залежить від провайдера. Хмарна модель – вибір для продакшену, де важлива якість, швидкість і підтримка української мови. Вона в десятки разів швидша, пише чистою мовою, не вигадує термінів. Але залежить від мережі, має rate limits, і всі дані йдуть на чужий сервер.

Висновок за завданням 3

Експеримент показав, що OpenAI-сумісний інтерфейс дозволяє перемикатися між локальною та хмарною моделлю, змінивши лише два значення у словнику провайдерів. Різниця в якості виявилася драматичною: хмарна модель на 120В параметрів дала точну відповідь чистою українською за 0,92 с, тоді як локальна 3В-модель – суржик із вигаданими термінами за 23,80 с. Швидкість також на користь хмарної: Groq використовує спеціалізоване обладнання LPU, яке перекриває навіть затримку мережі. Практичний висновок: локальну модель варто обирати там, де дані не можна передавати назовні, де потрібна робота без інтернету, або для навчання й прототипування, коли якість не критична. Вона безкоштовна, приватна й не залежить від провайдера. Хмарну – для продакшену, де важлива якість, швидкість і підтримка української мови. Вона в десятки разів швидша, пише чистою мовою, не вигадує термінів, але залежить від мережі, має rate limits, і всі дані йдуть на чужий сервер.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано заміри швидкості для двох провайдерів – локальної llama3.2:3b та хмарної openai/gpt-oss-120b – на трьох промптах різної довжини. Для кожного промπτ зроблено прогрів і три повтори з медіаною, щоб уникнути впливу холодного старту й випадкових викидів. Окремо виміряно холодний і теплий старт локальної моделі.

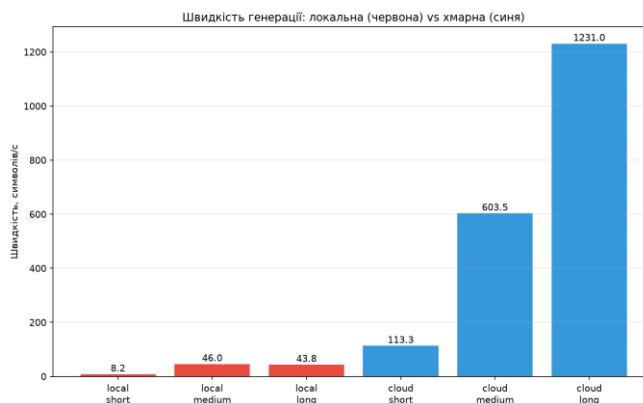
Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього,с	Симв./с
локальна	llama3.2:3b	короткий	2,154	2,887	8,2
локальна	llama3.2:3b	середній	2,266	8,888	46,0
локальна	llama3.2:3b	довгий	2,282	39,145	43,8
хмарна	openai/gpt-oss-120b	короткий	0,357	0,377	113,3
хмарна	openai/gpt-oss-120b	середній	0,394	0,663	603,5
хмарна	openai/gpt-oss-120b	довгий	0,354	2,834	1231,0

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	16,20	2,70
Всього, с	16,83	3,00

Крок 5. Побудувати стовпчикову діаграму (matplotlib): швидкість у символах за секунду для всіх шести комбінацій «модель × промπτ». Можете запитати у LLM - як написати код для генерації стовпчикової діаграми.



Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: У хмарної. Причина: Groq використовує LPU-чипи з наднизькою латентністю, і мережева затримка виявляється меншою за час, який локальна CPU-модель витрачає на токенизацію й обробку промпта.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні – поясніть.

Відповідь: Так, збігається, хмарна модель виграла і за TTFT, і за швидкістю генерації. Але теоретично вони можуть не збігатися: TTFT залежить від обробки промпта й мережі, а швидкість генерації від обчислювальної потужності. У моєму випадку Groq виграв в обох компонентах.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: На короткому промпті різниця менша, бо значну частину часу займає TTFT – фіксована величина, яка майже однакова для обох моделей за структурою, але в локальній вона в 6 разів більша. На середньому й довгому промптах різниця стабілізується на рівні 14, бо домінує генерація, де різниця в швидкості стала вирішальною.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Три повтори були стабільними – медіана близька до всіх трьох значень. Але одного запуску недостатньо: модель недетермінована, і перший запуск може бути спотворений холодним стартом. Медіана трьох запусків стійкіша до викидів.

5. Чому холодний старт повільніший за теплий?

Відповідь: Холодний старт включає завантаження моделі з диска в RAM (2 ГБ для llama3.2:3b) – це 13,5 с чистої затримки. Теплий старт – модель уже в пам'яті, залишаються лише токенизація й обробка промпта.

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що фізично відбувається під час TTFT – часу до першого токена – і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT модель токенизує промпт, проганяє його через усі шари і формує перший токен. Це принципово інша фаза, ніж генерація: prefill обробляє весь вхідний контекст одразу, а генерація видає токени по одному. TTFT критичний для UX, а швидкість генерації – для довгих відповідей.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Хмарна виграла в обох, але теоретично TTFT залежить від мережевої затримки + черги на сервері, а швидкість генерації – від обчислювальної потужності. Якщо хмарний сервер перевантажений, TTFT може бути гіршим, але потім генерація все одно швидша за локальну.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт – це завантаження ваг моделі з диска в оперативну пам'ять. Для llama3.2:3b це 2 ГБ, які треба прочитати й розмістити в RAM. Після цього модель залишається в пам'яті і наступні запуски йдуть по теплому шляху.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Модель недетермінована і час генерації залежить від довжини відповіді, яка щоразу різна. Медіана стійкіша до викидів, ніж середнє, і показує «типову» продуктивність. Одного запуску недостатньо, бо він може випадково потрапити на холодний старт або аномально довгу відповідь.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Prefill-фаза обробляє всі токени промпта паралельно через матричні множення, а генерація – послідовно, по одному токеноу за крок. Тому обробити 100 токенів промпта швидше, ніж згенерувати 100 токенів відповіді, навіть якщо обчислювальна складність схожа.

Для інтерактивного чат-бота найважливіший TTFT – саме він визначає, чи відчує користувач «зависання». Якщо $TTFT < 1$ с, інтерфейс здається миттєвим. Другорядний – загальний час: якщо відповідь коротка, вона догенерується, поки користувач читає перші слова. Groq з TTFT 0,35 с ідеально підходить для чат-ботів. Для фонові обробки великих обсягів тексту TTFT не критичний – ніхто не чекає в реальному часі. Тут важлива загальна пропускна здатність і вартість. Локальна модель може бути вигіднішою, якщо обсяги великі й приватність важлива, навіть попри повільність. Хмарна – якщо потрібна швидкість і якість.

Висновок до завдання 4

Експеримент показав, що затримка відповіді складається з двох різних величин – TTFT і швидкості генерації – і вони можуть суперечити одна одній. Всупереч очікуванням, хмарна модель виграла по обох: TTFT 0,35 с проти 2,15 с у локальній (у 6 разів), а швидкість генерації – 113–1231 симв/с проти 8–46 (у 14–28 разів). Мережева затримка виявилася меншою за час обробки промпта локальною CPU-моделлю. Найяскравіше різниця проявилася на довгому промпті: локальна витратила 39,15 с проти 2,83 с у хмарної – у 13,8 разів. Окремо виміряно холодний і теплий старт: TTFT холодного – 16,20 с, теплого – 2,70 с, різниця 13,5 с – це чистий час завантаження моделі в RAM. Практичний висновок: для інтерактивних застосунків із довгими відповідями хмарна модель майже безальтернативна. Локальна підходить для коротких запитів, де TTFT ~ 2 с прийнятний, і де важлива приватність. Для агентів із багатьма послідовними викликами різниця стає ще драматичнішою: 8 викликів * 39 с = 5 хвилин у локальній проти 23 с у хмарної.

Завдання 5. Оцінювання якості моделей за рубрикою

Виконано прогони трьох моделей – локальної інструктивної llama3.2:3b, локальної міркувальної qwen3:4b та хмарної openai/gpt-oss-120b – за фіксованим набором із шести завдань. Оцінювання проводилося за трибальною шкалою (0 – не впорався, 1 – частково, 2 – виконано повністю) згідно з критеріями Таблиці 10

методички. Правильну відповідь на завдання 1 (рік відкриття пеніциліну) перевірено за авторитетним джерелом. Завдання 2 перевірено об'єктивним тестом через `json.loads()`.

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	1	2	2
2. Формат	1	1	2
3. Стиснення	0	1	2
4. Код	1	1	1
5.Логіка	0	2	2
6. Українська	0	2	2
Разом /12	3	9	11

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	так	ні	ні
2. Формат	так	так	так
3. Стиснення	ні	так	ні
4. Код	ні	так (таймаут)	ні
5.Логіка	так	ні	ні
6. Українська	так	ні	ні

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала хмарна (11/12), потім qwen3:4b (9/12), найменше – llama3.2:3b (3/12). Основна різниця виникла на завданнях, де потрібні або точні факти, або міркування, або якісна мова.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Розбіжності: llama3.2:3b – на завданнях 1, 2, 5, 6; qwen3:4b – на 2, 3, 4; хмарна – лише на 2. Це говорить про нестабільність llama3.2:3b: вона дає різні відповіді на той самий запит, що робить її ненадійною. qwen3:4b стабільніша, хмарна – найстабільніша.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Ні, жодна. Усі три моделі дали валідний JSON, але в різних формах: llama3.2:3b і qwen3:4b – у вигляді одного об'єкта (але з неправильним змістом), хмарна – у вигляді масиву. Ніхто не дотримався точної специфікації. Це підтверджує тезу методички: промпт – ненадійний спосіб керувати форматом.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, у llama3.2:3b – постійно, у qwen3:4b – один перехід на англійську в завданні 3 (прогін 2). У хмарної – жодних проблем, чиста українська.

Крок 7. Обов'язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести:

Обраний випадок: llama3.2:3b, завдання 5 (логіка), прогін 1.

- повний текст запиту: 40 пацієнтів, 18 з температурою, 15 з кашлем, 7 з обома. У скількох немає симптомів? Покажи хід розв'язання.

- повний текст відповіді, без скорочень:

«Давайте використовуватимемо принцип "всі пацієнти" або "всі пацієнти є" щоб спростувати кількість пацієнтів, які не мають симптомів.

1. Patients з температурою - 18

2. Patients з кашлем - 15

3. Patients з обома - 7

Побачивши, що п'ять пацієнтів з обома мають симптоми, ми можемо припустити, що п'ять пацієнтів з обома не мають симптомів.

4. Patients з обома без симптомів - 7

Тепер, якщо ми зібратимо всі пацієнтів, які не мають симптомів, то ми отримаємо:

5. Patients без симптомів - 7 (з обома) + 7 (всі інші пацієнти, які не мають жодного симптому) = $7 + 33 = 40$

Всього у нас 40 пацієнтів, 7 з обома не мають симптомів, а 33 з інших пацієнтів не мають жодного симптому.

Итак, 33 пацієнта не мають жодного симптому.»

- яка правильна відповідь мала бути: 14 пацієнтів не мають жодного симптому
- пояснення, чому так сталося: Модель не зрозуміла формулу включення-виключення і замість неї почала міркувати наосліп: спочатку припустила, що 7 пацієнтів з обома симптомами не мають симптомів, потім додала $7 + 33 = 40$ і оголосила, що 33 пацієнти без симптомів. Це класична галюцинація в багатокроковому міркуванні: модель не має внутрішнього механізму перевірки, а просто генерує правдоподібний текст. Додатково суржик («Patients», «Всього», «Итак», «припустинити») показує, що модель «думає» російською, а «відповідає» українською.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п'ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Автоматичні метрики, LLM-as-a-Judge, людське оцінювання за шкалою Likert, бенчмарки, А/В-тестування в продакшені. Рубрика – найпростіший і найпрозоріший метод для невеликих наборів задач.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Модель навчена на мільйонах прикладів, де JSON обгорнутий у markdown-блоки. Ця закономірність настільки сильна, що перемагає пряму інструкцію. Модель знає краще, як має виглядати правильно оформлений JSON. Це фундаментальна проблема: промпт – ненадійний спосіб керувати форматом.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Бо в навчальному корпусі української мови на порядки менше, ніж англійської, а російської – більше, ніж української. Модель зривається на домінуючу мову або змішану. Малі моделі особливо вразливі, бо мають менше параметрів для запам'ятовування рідкісних мовних патернів.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Модель недетермінована: на кожному кроці вона семплить токен з розподілу ймовірностей, а не бере найімовірніший. Навіть при `temperature=0` можливі варіації через особливості обчислень із плаваючою комою. Це означає, що оцінка за одним прогоном ненадійна потрібно мінімум два.

5. Що таке галюцинація мовної моделі і чому модель радше вигадас факт, ніж визнає незнання?

Відповідь: Галюцинація – це генерація правдоподібного, але вигаданого тексту. Модель не має механізму «я не знаю» – вона завжди видає найімовірніший токен. Якщо факту немає в навчальних даних, статистично правдоподібним виявляється вигаданий варіант. Визнання незнання вимагало б окремого тренування.

Оцінка за рубрикою виявилася надійною: три моделі отримали чітко розділені бали (3, 9 і 11 із 12), і це конкретні числа, які можна перевірити й відтворити. Єдина проблема виникла із завданням 2, бо буквальный критерій Таблиці 10 каже «`json.loads()` парситься – 2 бали», і формально всі три моделі отримують по 2. Але зміст JSON у локальних моделей – галюцинація (у `llama3.2:3b` «палітка на Меркурія», у `qwen3:4b` «electromagnetic induction» замість пеніциліну), тому я оцінювала і формат, і зміст: локальні отримали 1 бал, хмарна – 2. Це доводить, що малий розмір сам по собі не вирок, але reasoning-модель обов'язкова для багатокрокових задач. Для українськомовного тексту чисту мову дали обидві моделі – і `qwen3:4b`, і хмарна. Я обрала б `qwen3:4b`, якщо пріоритет – приватність, безкоштовність і робота без інтернету: вона пише чисто українською, стабільно розв'язує логіку і не галюцинує факти. Хмарну обрала б, якщо пріоритет –

максимальна якість і швидкість: вона єдина дала правильний JSON у завданні 2 і відповідає в десятки разів швидше. Llama3.2:3b непридатна для таких задач незалежно від пріоритетів – вона пише суржиком, галюцинує факти й розходиться між прогнами.

Висновок для завдання 5

Експеримент показав, що оцінювання за рубрикою дає чіткий, відтворюваний профіль моделі: хмарна – 11 із 12, qwen3:4b – 9 із 12, Llama3.2:3b – 3 із 12. Основна різниця виникла на завданнях, де потрібні точні факти, багатокрокове міркування і якісна мова. Llama3.2:3b провалила всі три: вигадувала роки, не розв'язала логічну задачу і писала суржиком. qwen3:4b впоралась із логікою та мовою ідеально, але провалила формат і частково стиснення. Хмарна – єдина, хто дав правильний факт у JSON, але й вона порушила структуру, повернувши масив замість одного об'єкта. Практичний висновок такий: Llama3.2:3b не можна використовувати для задач, де потрібні факти, логіка або українська мова; qwen3:4b – оптимальний вибір для локальної роботи з українськомовним текстом (але дуже повільна для CPU); хмарна для того, де потрібна максимальна якість і швидкість. Головний висновок для подальших лабораторних: розмір моделі не вирішує все, бо qwen3:4b утричі краща за Llama3.2:3b завдяки reasoning-тренуванню, а хмарна виграла лише на 2 бали, бо навіть великі моделі ігнорують формат. Для агента, який викликає інструменти через структуровані дані, найкритичніше – завдання 2, і жодна модель не виконала його ідеально. Це підтверджує, що промпт – ненадійний спосіб керувати форматом, і для продакшену потрібні програмні обмеження, а не покладання на текст інструкції.

Завдання 6. Керування випадковістю: temperature і top_p

Виконано серію експериментів із локальною моделлю Llama3.2:3b через Ollama. Для творчого завдання (назва платформи взаємного навчання студентів) і фактичного питання (рік відкриття пеніциліну Флемінгом) зроблено по 5 запусків

при різних значеннях temperature і top_p. Ключова метрика для творчого завдання – кількість унікальних відповідей із п'яти; для фактичного – кількість правильних. Окремо перевірено відтворюваність: 10 запусків фактичного питання при temperature = 0.0. Правильна відповідь на фактичне питання: пеніцилін відкрито 1928 року Олександром Флемінгом.

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	5	-
temperature	1.5	5	2
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так/ні)	так

Крок 7. Відповісти на питання (по 1–2 речення):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для структурованих даних потрібна максимальна передбачуваність – щоб той самий запит давав той самий JSON. Експеримент це підтвердив: при temp=0.0 фактична відповідь була однаковою всі 10 разів.

2. Яке – генератору варіантів заголовків?

Відповідь: Для творчих задач потрібна різноманітність. При $temp=0.0$ модель видає один і той самий список, при 0.7 – 5 унікальних, при 1.5 – теж 5, але серед них можуть бути безглузді.

3. Чим вплив top_p відрізняється від впливу $temperature$ за характером, а не за силою?

Відповідь: $temperature$ змінює форму розподілу – сплющує або загострює ймовірності всіх токенів. top_p відсікає хвіст – залишає лише ті токени, що разом набирають $p\%$ ймовірності. У експерименті $top_p=0.1$ дала 1 унікальну відповідь, а $temperature=0.0$ – 2 унікальних. top_p сильніше обмежує різноманітність.

4. Чи означає $temperature=0$ повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: У експерименті – так, 1 унікальна з 10. Але методичка попереджає: гарантії немає, бо обчислення з плаваючою комою недетерміновані, а на GPU додається паралелізація. У моєму випадку модель переважно на CPU – тому стабільно.

5. Наведіть найбезглуздішу відповідь, яку модель видала при $temperature=1.5$, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: $temperature=1.5$: унікальних 5 з 5

- Обіємо запропонувати такі плани з відсотної процентки:

1. ****Сіместероносентрально****: Студенті на 100% взяли участь в онліній строчкі мадбйтере.

2. ****Троесентрально-експериментальні****: Студенті на 66,7% взяли участь у онліній строчкі мдбйтері. Цимі обйтімі сантайся вшанованмйя екперменмвмей.

3. ****Дуесентоносентрально****: Студенті делять сеймс у online навчичальї.

Це суміш декількох мов і вигаданих слів. З'явилася тому, що при $temperature=1.5$ розподіл сплющується: малоїмовірні токени отримують шанс, і модель обирає випадкові комбінації символів.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Відповідь: `Temperature` ділить логіти на значення T . При $T < 1$ розподіл загострюється (найімовірніший токен домінує), при $T > 1$ – сплющується. При $T = 1$ – вихідний розподіл моделі.

2. Чим `top_p` відрізняється від `temperature` – обидва впливають на різноманітність, але за яким механізмом?

Відповідь: `top_p` сортує токени за ймовірністю й залишає лише ті, що разом набирають p . Решта відкидається до семплінгу. `Temperature` ж змінює самі ймовірності, не відкидаючи жодного токена. Тому `top_p` обмежує простір вибору, а `temperature` – форму розподілу.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: На практиці зазвичай налаштовують щось одне: або `temperature`, або `top_p`. Одночасна зміна ускладнює розуміння, що саме вплинуло. Стандарт: `temperature` для творчих задач, `top_p` для контролю «хвоста».

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Бо обчислення з плаваючою комою недетерміновані: порядок операцій, паралелізація на GPU, округлення можуть дати трохи різні логіти. У експерименті на CPU – 10/10 однакових, але на GPU могло б бути інакше.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для структурованих даних – `temperature = 0.0–0.2`. Для творчого тексту (назви, слогани, історії) – `temperature = 0.8–1.2`. Для чат-ботів загального призначення – `temperature = 0.7`. Для коду – `0.0–0.3`

Для наступних лабораторних, де модель має повертати структуровані дані, я оберу `temperature = 0.0` – експеримент підтвердив, що це дає ідеальну передбачуваність. При `temperature = 1.5` модель видала 3 з 5 неправильних років,

тобто випадковість тут працює проти. Налаштування за замовчуванням (0.7–1.0) для структурованих даних непридатне: модель може вигадати зайве поле або зламати формат, тому $temperature = 0$ треба ставити явно. Цікаво, що при $temperature = 0.0$ творче завдання дало не 1, а 2 унікальних відповіді – той самий список платформ, але з різними вступними фразами. Тобто set – груба метрика: вона ловить різницю у формулюваннях, а не в сенсі. А $top_p = 0.1$ дала лише 1 унікальну з 5 – сильніше обмеження різноманітності, ніж $temperature = 0.0$. Це підтверджує, що параметри діють різними механізмами: $temperature$ змінює форму розподілу, а top_p відсікає хвіст.

Висновок до завдання 6

Експеримент показав, що $temperature$ і top_p – регулятори випадковості, а не «розумності». При $temperature=0.0$ модель дає передбачувані відповіді: на фактичне питання – 1928 п'ять разів, на відтворюваність – 10/10 однакових. При $temperature=1.5$ модель вигадує: 3 з 5 років неправильні, а творчі відповіді перетворюються на безглуздий набір слів. top_p діє інакше: при $top_p=0.1$ – 1 унікальна з 5, при $top_p=1.0$ – 5 унікальних. Тобто top_p обмежує простір вибору, а $temperature$ – форму розподілу. Практичний висновок: для структурованих даних – $temperature = 0.0$. Для творчих задач – $temperature = 0.8–1.2$. Для чат-ботів – 0.7. Налаштування за замовчуванням (0.7–1.0) не підходить для агентів: воно дає випадковість, яка ламає формат.

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	8-46 симв/с	~15-25 симв/с	113-1231 симв/с
TTFT	2,15-2,28	1-3 хв (thinking)	0,35-0,39
Якість, балів /12	3	9	11
Стабільність формату (завдання 2)	нестабільна (галюцинації)	нестабільна (не той факт)	відносно стабільна

Потрібні ресурси	2Гб RAM, CPU+GPU	3,2Гб RAM, CPU+GPU	GPU/CPU/RAM (на сервері провайдера)
Вартість	безкоштовно	безкоштовно	безкоштовно (з лімітами)
Куди йдуть ваші дані	нікуди(локально)	нікуди(локально)	на сервер Groq
Працює без інтернету	так	так	ні
Коли обирати саме її	для прототипів, де якість не критична	для локальної роботи з укр. текстом, коли важлива приватність	максимальна якість і швидкість

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.